

Test Plan Report

1. Introduction

This test plan describes how the graduation project titled AI-Powered Mock Interview System will be tested.

The purpose of this test plan is to verify that the developed system functions correctly, meets all specified functional and non-functional requirements, and produces reliable and consistent results.

The testing process aims to identify potential errors, validate system behavior under different scenarios, and ensure system stability before final deployment.

2. Project Scope

2.1. Project title

AI-Powered Mock Interview Platform with Mode-Driven Interviewer Behavior and Evidence-Based Feedback

2.2. Brief description of the system:

The AI-Powered Mock Interview System is a web-based application designed to simulate realistic HR and Technical interview sessions using artificial intelligence technologies. Users configure interview parameters such as role, domain, difficulty level, and interview mode (Supportive or Neutral).

The system conducts voice-based interviews using Speech-to-Text (STT) and Text-to-Speech (TTS) technologies to enable real-time interaction. In addition, the system performs mandatory camera-based behavioral analysis during the interview to evaluate attention level, facial expressions, and non-verbal cues.

At the end of each session, the system generates a comprehensive feedback report that combines content evaluation, communication performance, and behavioral analysis results with personalized improvement suggestions.

2.3. Main functionalities of the system:

- Interview type selection (HR or Technical)
- Interview configuration (role, domain, and difficulty level)
- Supportive and Neutral interview modes
- Real-time voice-based interview using STT and TTS
- AI-generated dynamic and context-aware interview questions
- Time-bound prompting for long responses
- Off-topic redirection and uncertainty handling (Supportive mode)
- Mandatory camera-based behavioral analysis during the session

- Generation of a comprehensive feedback report including:
 - Content evaluation
 - Communication performance
 - Behavioral analysis results
 - Personalized improvement recommendations

3. Features to Be Tested

The following features will be tested:

- Interview type selection (HR / Technical)
- Interview configuration (role, company, domain, difficulty)
- Supportive and Neutral modes
- Voice interaction (STT & TTS)
- AI-based question generation
- Time-bound prompting
- Off-topic redirection (Supportive mode)
- Uncertainty handling with hints (Supportive mode)
- Feedback report generation
- Behavioral signal processing
- System responsiveness and latency
- Privacy and consent handling

4. Testing Methodology

4.1. Unit Testing

Goal: Verify that each backend and frontend package works correctly in isolation by testing internal logic, validation rules, and data processing using mocked external dependencies.

Unit tests focus on verifying the following aspects of the system:

- Validation of interview configuration
- Guardrail rule evaluation (time limits, off-topic detection, uncertainty detection)
- Behavioral signal computations
- Feedback report generation logic
- Correct invocation of STT and TTS services
- Request validation and error handling
- Frontend component logic and state transitions

These tests ensure that individual modules behave correctly before system integration.

Backend Packages (server/src)

- **domain/ (models, enums, dto)**

Validate data structures, validation rules, DTO mapping, and serialization/deserialization.

- **orchestration/ (GuardrailsEngine, InterviewFlowPolicy)**

Unit test guardrail rules: time-bound checks, uncertainty phrase detection, off-topic detection heuristics.

- **orchestration/ (BackendOrchestrator, InterviewFlowPolicy)**

Unit test decision logic for turn flow (neutral vs supportive), question progression, and intervention selection using mocked AI and policy outputs.

- **services/ai/ (AIServiceGateway, OpenAIClientAdapter, OpenAiRealtimeGateway)**

Unit test gateway/adapters for correct request formatting, error handling, retries/backoff, and safe fallbacks (no real external calls).

- **services/analysis/ (BehaviorAnalyzer, AudioSignalProcessor, VisionSignalProcessor, TranscriptEvaluator, PythonAnalysisClient)**

Unit test behavioral signal computations (filler rate, pause rate, speaking rate, engagement proxies) using synthetic transcript/video summaries.

- **api/controllers/ & persistence/storage/ (FileReportArchive)**

Unit test report assembly, scoring, recommendation generation, and graceful degradation when signals are missing.

- **api/controllers/ (SessionController, ConsentController, RealtimeController, ReportController)**

Unit test request validation and response/error mapping (400/500) with mocked orchestration layer.

Frontend packages (client/src)

- **pages/**

Unit test page-level logic (button actions, navigation triggers) with mocked services.

- **components/**

Unit test UI components rendering and props/state behavior.

- **lib/ (realtimeClient) & services/**

Unit test API client request/response mapping, WebRTC connection handling, and error handling with mocked network layer.

- **lib/ & app/ (state)**

Unit test state transitions (setup → started → turn submitted → report ready).

4.2. Integration Testing

Goal: Verify that different system components interact correctly and that data flows properly across system boundaries.

Integration testing validates:

- API request lifecycle (routes → controllers → orchestration)
- interaction between orchestration logic, guardrails, and AI services
- Audio input → STT → guardrail evaluation → LLM question generation → TTS (Realtime API) output
- feedback generation pipeline using session data and analysis outputs (Python API)
- frontend-backend communication and UI updates

These tests ensure correct collaboration between system components.

Backend integrations (server/src)

- **api/routes + api/controllers + orchestration/BackendOrchestrator**

Validate endpoint-to-service wiring, request lifecycle, and correct status codes/payloads.

- **orchestration/BackendOrchestrator + orchestration/GuardrailsEngine + services/realtime**

Validate STT → guardrail evaluation → LLM/TTS decision pipeline using mocked OpenAI realtime provider.

- **orchestration/BackendOrchestrator + services/analysis + persistence/storage/FileReportArchive**

Validate end-of-session report generation using real session data + synthetic signals via PythonAnalysisClient.

Frontend integrations (client/src)

- **pages/ + lib/realtimeClient + app/**

Validate “record/submit/receive output” loop behavior with mocked backend/OpenAI responses.

- **pages/ + components/**

Validate correct UI updates when transcript/intervention/next question is received.

4.3. System Testing

Goal: Validate the complete system as a black-box application and ensure that the entire interview workflow functions correctly from the user’s perspective.

System tests verify the full workflow:

Setup → Consent → Interview Session → Question/Answer Loop
→ Guardrail interventions → End Session → Feedback Report

Special focus areas include:

- interview mode behavior (Neutral vs Supportive)
- voice interaction using STT and TTS (OpenAI Realtime API)
- correct UI navigation between setup, interview, and feedback screens
- microphone & camera permission enforcement
- behavioral analysis (Audio & Vision signal processing) inclusion when camera access is granted

If camera access is denied or behavioral analysis fails, the system must gracefully degrade and still produce a valid feedback report.

Packages involved: all frontend (pages, components, lib, app) and all backend (api, orchestration, services, domain, persistence).

4.4. Performance Testing

Goal: Validate responsiveness and stability for live usage.

Performance focus by package:

- **services/realtime & services/ai**

Latency and reliability of AI calls (OpenAI WebSocket timeouts, rate limits) and fallback behavior.

- **orchestration/BackendOrchestrator**

End-to-end turn latency (input received → output decided).

- **services/analysis (PythonAPI & Node.js client)**

Report generation time after session end (audio processing & Llama3 analysis).

- **api/routes & api/controllers**

Endpoint throughput and concurrency behavior under load.

Metrics:

- turn latency (p95/p99)
- report generation time
- error rate
- retry counts
- concurrent session handling

4.5. User Testing

Goal: Validate usability and perceived value of the system with real users.

User test targets by package:

- **Frontend pages/components**

Clarity of guided flow, consent UX, interview screen usability, VoiceWaveCanvas responsiveness, feedback screen readability.

- **Backend orchestration + services/analysis**

Perceived coherence of interview flow and usefulness/actionability of feedback.

- **Privacy expectation checks**

Consent handling and no credential exposure.

Outputs: scenario-based sessions (Neutral and Supportive), user survey + qualitative feedback, usability issues mapped to NFR/PSR.

5. Test Environment

5.1. Hardware:

- Laptop / PC
- Microphone
- Minimum 8 GB RAM
- 4-core CPU (recommended for audio/video processing)
- Webcam with at least 720p resolution
- Stable internet connection (recommended ≥ 10 Mbps)

5.2. Software:

- Windows 11
- Node.js (Backend API Server)
- React + TypeScript (Frontend)
- Python (AI processing services for speech and behavioral analysis)
- OpenAI API (LLM, STT, TTS services)
- Browser (Chrome) (WebRTC and MediaDevices support)

6. Test Cases

Test data includes synthetic interview transcripts, scripted user responses, sample audio inputs, and simulated behavioral signals for controlled testing of guardrails and feedback generation logic.

Test ID	Description	Input(s)	Expected Output	Result
UTC-01	Validate interview configuration (type, role/context,	Config DTO with valid/invalid combinations	Valid config accepted; invalid config returns validation error messages	passed

	domain, difficulty, mode)			
UTC-02	Guardrail: time-bound prompting triggers when answer exceeds allocated window	Transcript duration + elapsed response time > limit	GuardrailsEngine triggers session update prompting AI to intervene with TIME_BOUND_PROMPT guidance	
UTC-03	Supportive: off-topic detection triggers gentle redirect	Question + transcript that drifts off-topic	GuardrailsEngine updates session instructions to redirect candidate back to topic	
UTC-04	Supportive: uncertainty detection triggers clarifying sub-questions/hints	Transcript contains “I don’t know / I’m not sure” (and TR equivalents)	TurnOutput returns supportive follow-up question/hint (no direct answer reveal)	passed
UTC-05	Feedback report aggregation works with missing behavioral signals (graceful degradation)	Session turns + only speech signals (no video)	Report generated; behavioral section partial; core feedback still produced	
UTC-06	Security: client contains no API key; AI calls routed only via backend gateway	Frontend bundle scan + server gateway mock	No API key found in client; server gateway invoked for AI calls	passed
UTC-07	STT transcription pipeline processes recorded responses	Recorded audio (.webm) + mock STT output	CandidateAudioTranscriber generates transcript and stores it per response	passed
UTC-08	Real-time audio response streaming	AI interviewer response during session	Streaming audio delivered through	passed

	works through WebRTC		WebRTC connection without errors	
ITC-01	Create session API returns sessionId with stored configuration	POST /api/session with config	200 OK + sessionId; server stores config	passed
ITC-02	Start session returns first question consistent with context/difficulty + audio output	POST /api/session/{id}/start	Question text matches context/difficulty; interviewer audio streamed via Realtime API	
ITC-03	Audio streaming pipeline: WebRTC audio → guardrails → next system action (neutral)	WebRTC audio stream from frontend	Transcript processed; guardrails evaluated; next interviewer response generated	
ITC-04	Supportive mode triggers redirect/uncertainty behaviors end-to-end	Session in SUPPORTIVE + off-topic/uncertainty audio	GuardrailsEngine triggers redirect or supportive follow-up through AI session update	passed
STC-01	End-to-end Neutral session completes within target 10–15 minutes and generates report	Full UI flow, neutral mode, typical user answers	Session completes; report shown; duration within 10–15 min target	
STC-02	End-to-end Supportive session: redirection + uncertainty assistance observed	Supportive mode + scripted off-topic + “I’m not sure” answers	Redirect and supportive guidance triggered; flow remains coherent	

STC-03	Consent flow: microphone permission required to start interview	Deny mic / allow mic	Mic denied ⇒ cannot start; mic allowed ⇒ session starts	passed
STC-04	UI navigation: Setup → Interview → Feedback screens	Complete run through UI	All required screens available and reachable; correct transitions	passed
STC-05	Behavioral analysis from audio signals included in report	Run session with recorded speech signals	Report includes audio-based behavioral signals (speech pace, pauses, filler rate)	passed
PTC-01	Turn responsiveness (latency) meets real-time conversation expectation	Run scripted turns; measure stop-speaking → system response	p95 turn latency ≤ 1.5 seconds between user speech end and next system response	
PTC-02	Feedback generation time is acceptable after session ends	End session; measure /end response time	Feedback report generated within 10 seconds after interview completion	
PTC-03	Reliability under partial availability: degrade gracefully if behavioral analysis fails	Force BehaviorAnalyzer(audio) failure	Interview completes; report generated using remaining signals	
PTC-04	Limit redundant external AI calls to control cost	Run session; count LLM/STT/TTS calls	AI calls per turn/session remain within defined cap	
UAT-01	Usability: guided flow starts interview in few steps	Real user performs setup	Users can start session quickly; low confusion; no unnecessary steps	passed

UAT-02	Feedback quality is constructive/coaching-oriented (psychological safety)	Real user reviews report	Feedback tone is supportive; no humiliating language; actionable tips provided	
--------	---	--------------------------	--	--

7. Risks and Limitations

Despite careful system design and testing, several technical risks and limitations may affect the performance and reliability of the AI-Powered Mock Interview System.

Speech Recognition Accuracy (STT Limitations)

The system relies on Speech-to-Text (STT) technologies to transcribe user responses during interview sessions. Although the primary interview language is Turkish, users may frequently use technical English terms such as programming languages, frameworks, or algorithm names.

STT models may occasionally misinterpret mixed Turkish-English speech or incorrectly transcribe technical terminology. These transcription inaccuracies may influence the content evaluation process and reduce the reliability of generated feedback.

Audio Noise and Microphone Quality

Background noise, low-quality microphones, or unstable audio input can negatively affect speech recognition accuracy. Environmental noise may cause partial or incorrect transcription of the user's responses, which can impact the system's interpretation of answers.

Face Detection and Lighting Conditions

Behavioral analysis relies on webcam input to detect non-verbal signals such as facial expressions, attention level, and head movements.

However, poor lighting conditions, low camera resolution, or camera positioning issues may cause face detection errors or reduce the accuracy of behavioral signal extraction.

Audio and Video Processing Latency

The system performs multiple computational processes including speech transcription, behavioral signal extraction, and feedback generation. Processing audio and video data may require significant computational resources.

As a result, the system may experience delays in response generation or feedback report creation, especially on devices with limited hardware performance.

LLM Turkish Language Support

The system uses large language models (LLMs) to generate interview questions and feedback reports. Although modern LLMs support Turkish, the language support may not always be as robust as English.

This limitation may occasionally result in less natural phrasing, reduced contextual accuracy, or imperfect interpretation of user responses.

Internet Connectivity and External API Dependency

The system relies on internet connectivity and external AI services such as LLM, STT, and TTS APIs. Network instability, API latency, or service interruptions may affect system responsiveness and real-time interaction during interview sessions.

8. Conclusion

This test plan provides a structured approach for validating the AI-Powered Mock Interview System.

By applying multiple testing methodologies including unit testing, integration testing, system testing, performance testing, and user testing, the system's functionality, reliability, and usability are evaluated comprehensively.

The defined test cases ensure that both functional and non-functional requirements are properly validated under realistic operating conditions. Performance testing evaluates system responsiveness, while user testing assesses usability and the effectiveness of generated feedback.

Although certain limitations exist, such as speech recognition inaccuracies, environmental noise, lighting conditions affecting behavioral analysis, processing delays, and dependency on external AI services, the testing strategy helps ensure that the system operates reliably and provides meaningful interview practice for users.

Overall, the implemented testing process supports the development of a stable AI-driven mock interview platform capable of simulating realistic interview interactions and delivering constructive feedback to help users improve their interview performance.

9. Requirement Traceability Matrix

Requirement ID	Requirement Description	Related Test Case(s)
FR-01	Select interview type (HR/Technical).	UTC-01, ITC-01, STC-01
FR-02	Provide role and context info.	UTC-01, ITC-01, STC-01
FR-03	Select domain and difficulty.	UTC-01, ITC-01, ITC-02
FR-04	Support Supportive and Neutral modes.	UTC-01, ITC-04, STC-02
FR-05	Run voice interview (10–15 min).	STC-01
FR-06	Capture speech and transcribe (STT).	UTC-07, ITC-03
FR-07	Generate questions and follow-ups.	ITC-02, ITC-03, ITC-04
FR-08	Enforce answer time limits.	UTC-02, ITC-03
FR-09	Supportive: redirect off-topic answers.	UTC-03, ITC-04, STC-02
FR-10	Supportive: help with uncertainty.	UTC-04, ITC-04, STC-02
FR-11	Generate feedback report with scores.	UTC-05, STC-01, STC-02
FR-12	Provide interviewer voice (TTS).	UTC-08, ITC-02

FR-13	Use camera (with consent) for signals.	STC-05
NFR-01	Simple guided user flow.	UAT-01, STC-04
NFR-02	Responsive live interaction.	PTC-01
NFR-03	Graceful degradation on failures.	UTC-05, PTC-03
NFR-04	Consent + no exposed API keys.	STC-03, UTC-06
NFR-05	Modular and maintainable design.	UTC-05
UIR-01	Setup screen available.	ITC-01, STC-04
UIR-02	Interview screen available.	ITC-03, STC-04
UIR-03	Feedback screen available.	STC-01, STC-04
HIR-01	Microphone integration (required).	STC-03, ITC-03
HIR-02	Camera integration	STC-05
SIR-01	External AI services integration	ITC-02, ITC-03
SIR-02	AI calls only via backend.	UTC-06
PR-01	Session time meets target.	STC-01
PR-02	Turn latency acceptable.	PTC-01
PR-03	Report generation fast enough.	PTC-02

PR-04	Degrade gracefully under issues.	UTC-05, PTC-03
PR-05	Bounded AI call rate (cost).	PTC-04